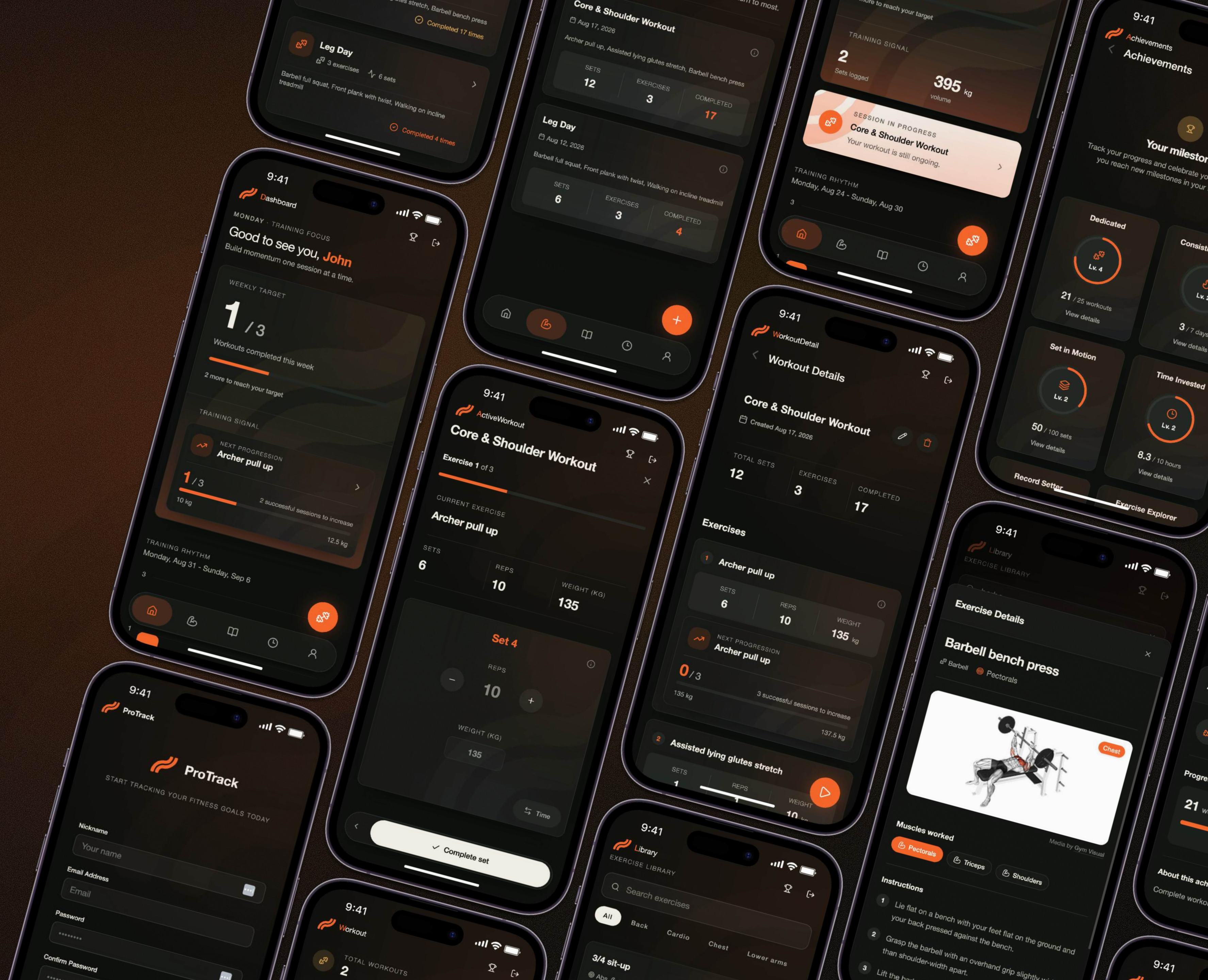




PRODUCT CASE STUDY · UX/UI · FRONTEND · BACKEND

DESIGNED & DEVELOPED BY
Manuel La Porta



The Problem

SOMETHING USEFUL FOR ME

"I started ProTrack because the tools I was using could record my workouts, but they didn't really help me decide what should happen next. I could see what weight I used last time and how many repetitions I completed, but deciding when to increase or decrease the load still depended on me remembering previous sessions and interpreting that data manually."

ProTrack is a fitness app that grows with you, helping you with workouts progression & overload.

I wasn't aiming for something popular or revolutionary, I wanted something that i could primarily use and recommend to my friends, gathering feedback; something that could help me in my day to day activities.

The result is a full-stack fitness application I designed and built where users build routines, complete one set at a time, and turn those sessions into useful feedback such as weekly progress, personal records, achievements, and automatic weight adjustments.

MY TECH STACK



Figma (Design)



TypeScript (FE/BE)



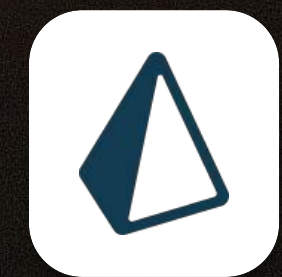
Vue 3 (FE)



Node & Express (BE)



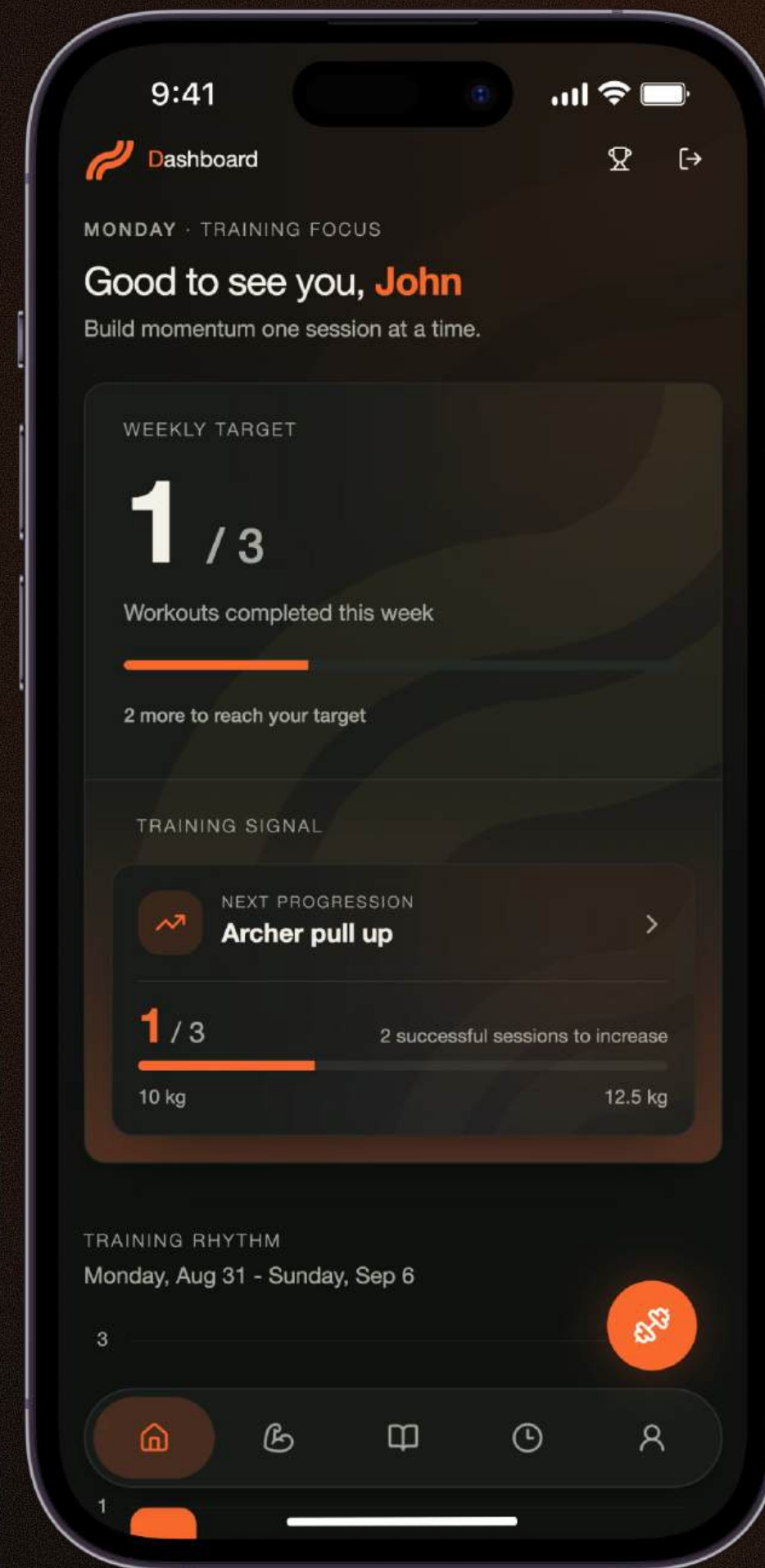
PostgreSQL (DB)



Prisma (ORM)

The Problem

SOMETHING USEFUL FOR ME



REGULAR TRAINING FLOW

PLAN

Choose exercises
and targets

TRAIN

Complete
the workout

LOG

Write down
sets and weight

PROGRESS

Assess based on
memory & assumptions

TRAINING USING PROTRACK

PLAN

TRAIN

LOG

PROGRESS

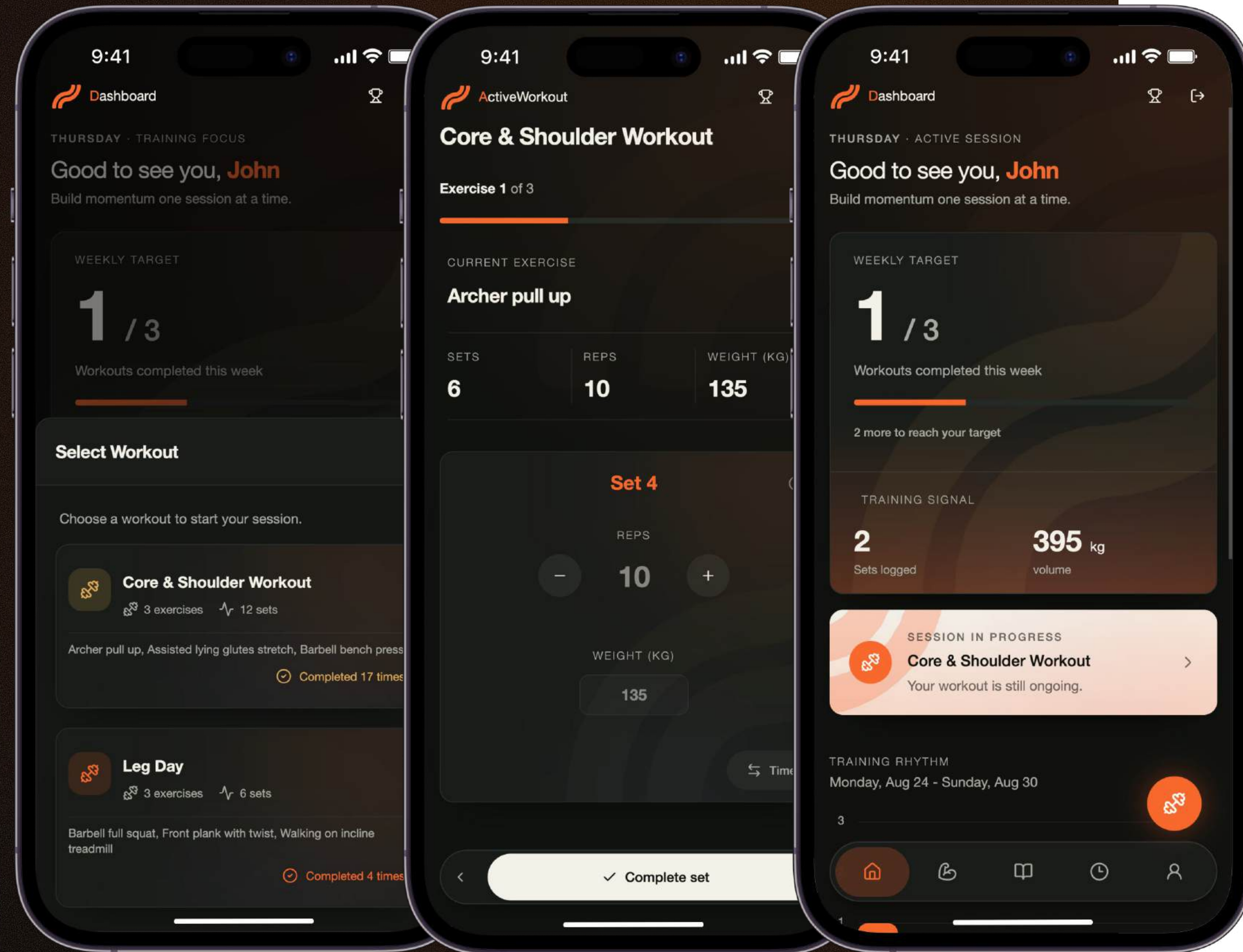


In a normal workflow, there is a gap between recording the workout and deciding what to do next. That decision usually happens manually.

With ProTrack, every completed set becomes part of a continuous feedback loop. The workout is logged, the performance is evaluated, the progression state is updated, and that information becomes visible again through the Dashboard and the next workout.

One simple flow

THE CORE EXPERIENCE IS ONLY THREE STEPS



The main flow of ProTrack is intentionally simple:

- 1 The user selects one of their saved workouts. Then the application guides them through the session one exercise and one set at a time.
- 2 Every completed set is saved while the workout is happening. (Nothing is lost!)
- 3 Once the session is completed, those results are evaluated against the targets of the workout, and the progressive overload counter for each exercise is updated. That updated state then follows the exercise into the next workout.

Creating a workout

BEFORE TRAINING, CREATE A PLAN

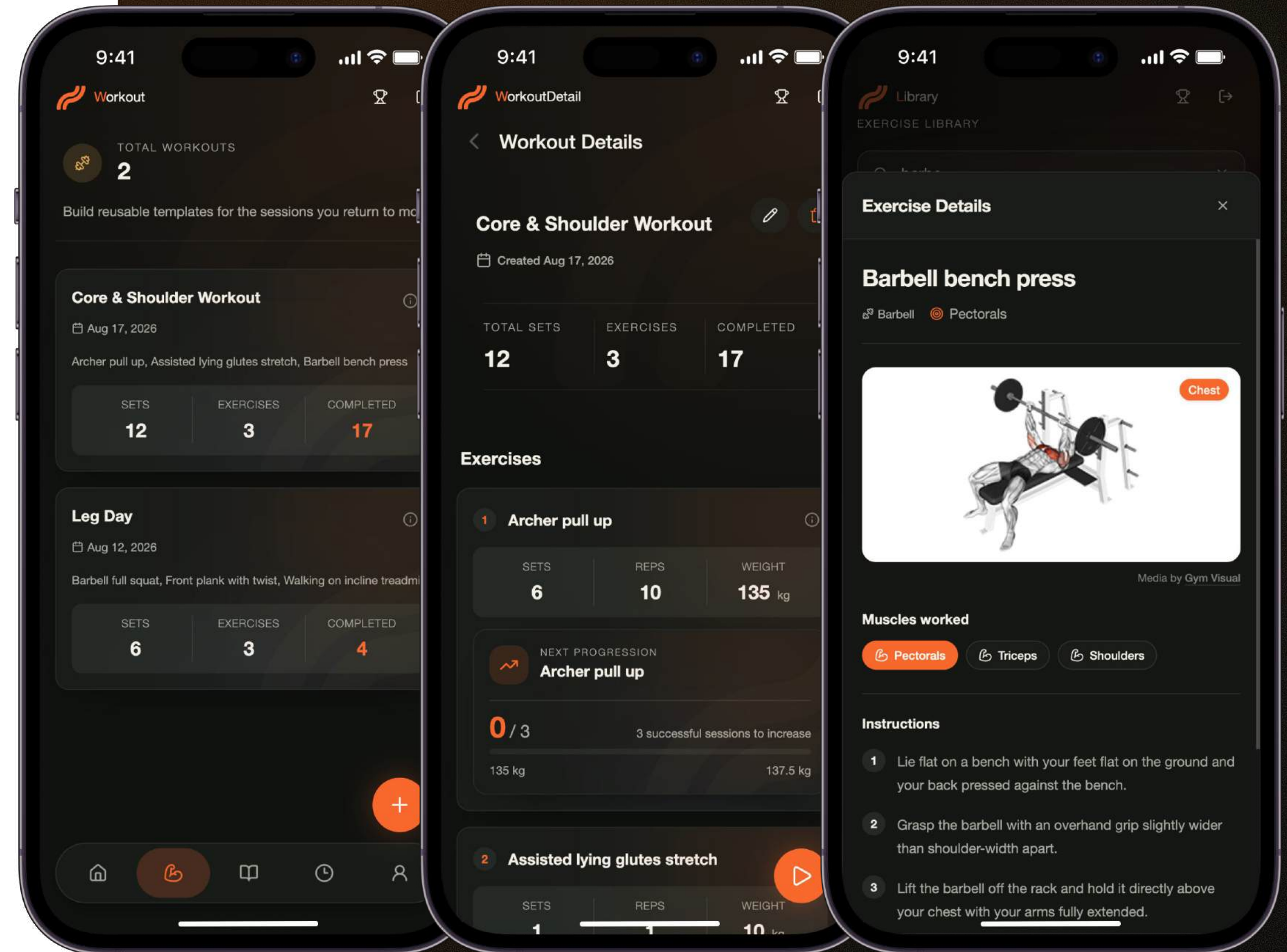
Before a workout can enter the main flow, the user creates it as a reusable routine, aka a **workout**.

They choose exercises suggested directly from the exercise library provided and define the target for each one: the number of **sets**, **repetitions**, **weight** in the **preferred units** (set in the Profile section).

That distinction is important because the workout template represents what the user intends to do.

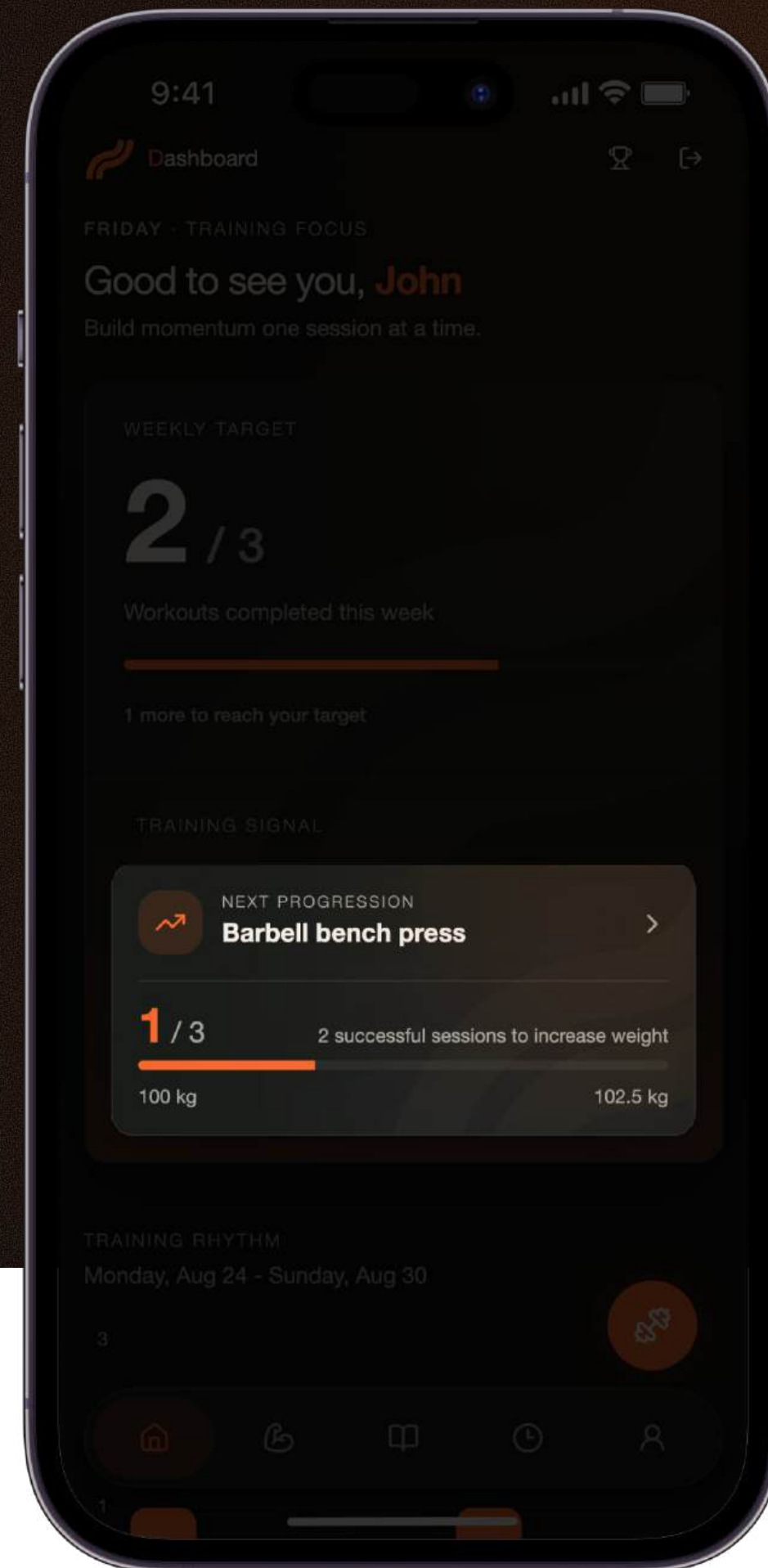
When they later start that workout, ProTrack creates an actual workout session representing what really happened.

This allows the same routine to be repeated while still keeping every training session separate.



Progressive Overload

A SAFE PROGRESSION OVER TIME



Progressive overload is **tracked individually** for every exercise.

Under each exercise card, the user can see its current progression state while training. If the exercise repeatedly meets its targets, the counter moves toward a weight increase.

The same logic works in the **opposite direction**.

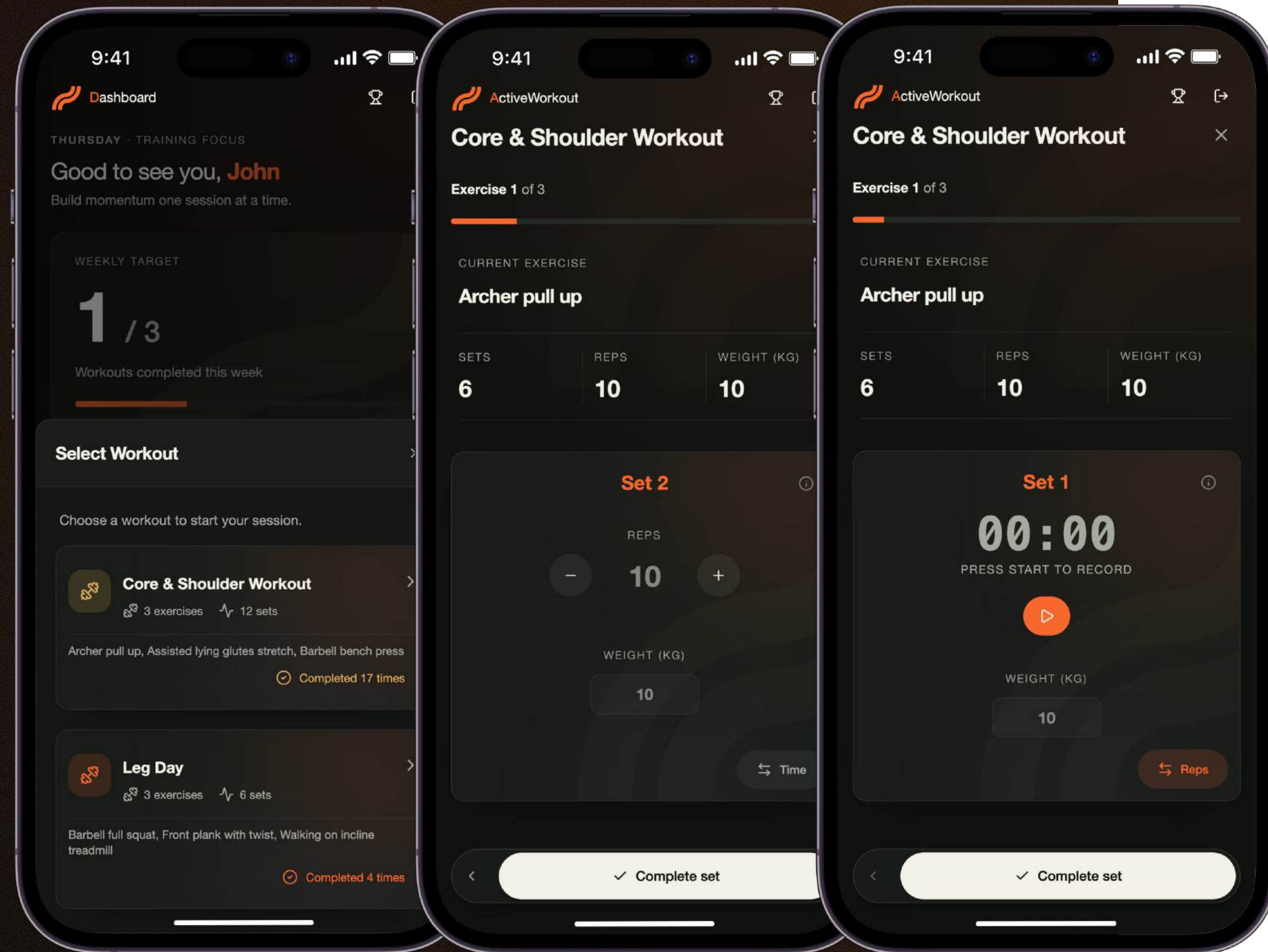
If the current load is consistently too demanding, the counter moves toward a weight decrease instead.

And because every completed set is logged, this logic is based on what actually happened during the workout rather than a manual button the user presses afterwards.

Progression in ProTrack doesn't assume that increasing weight is always the correct answer. The system can **adapt in both directions** safely based on athlete actual performance.

Progressive Overload

DIFFERENT EXERCISES, DIFFERENT INTERFACES



One UX problem I wanted to avoid was treating every exercise as if it worked the same way.

- For traditional **strength exercises**, the interface focuses on repetitions and weight.
- For **cardio** or **time based exercises**, repetitions are not the priority and the timer becomes the main control.

Weight can still remain available when it makes sense, for example with a weighted carry or a timed weighted exercise.

The screen therefore adapts to the exercise instead of making the user adapt to the interface.

Progressive Overload

UNRAVELING THE CORE APP FLOW

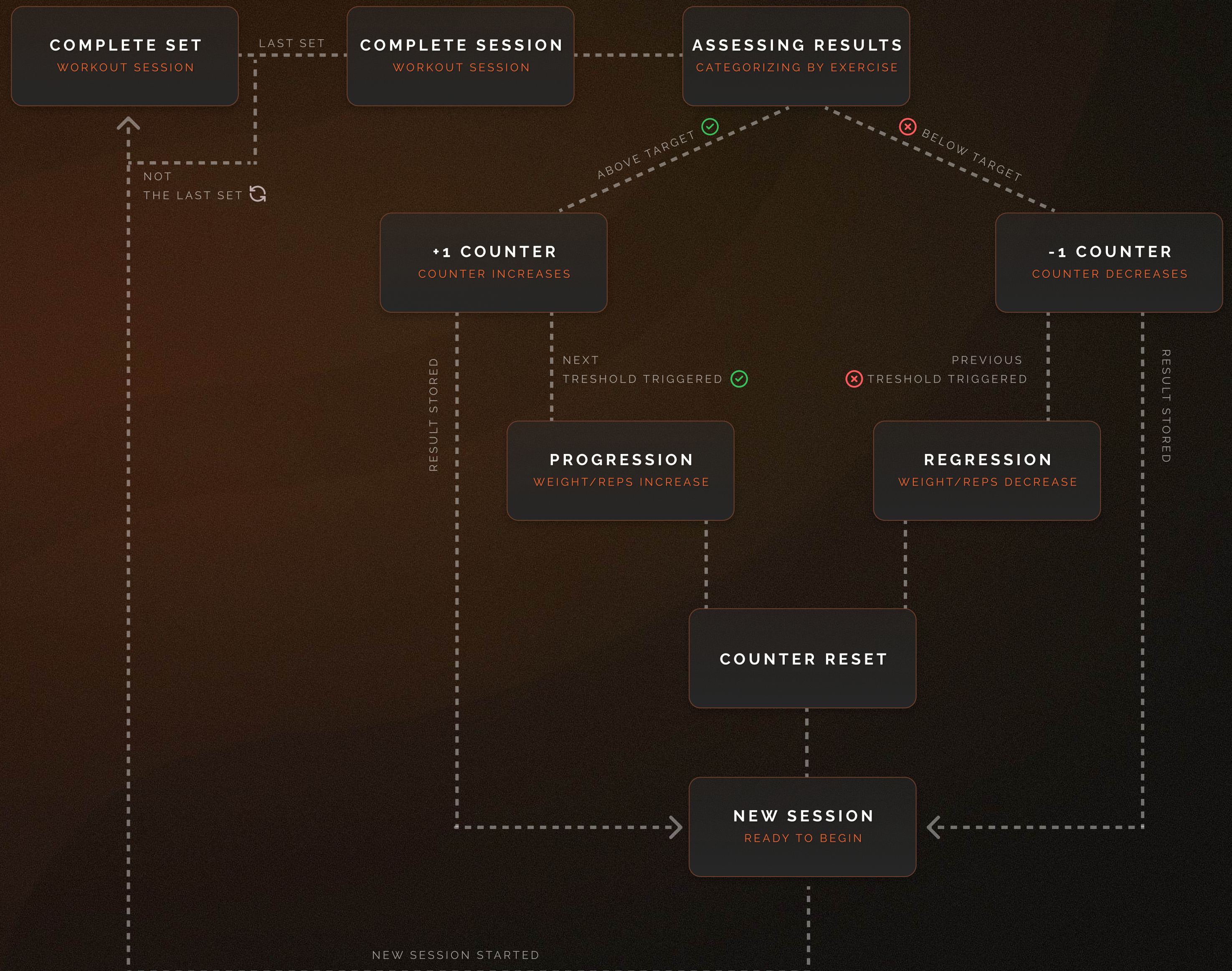
Every set is stored as the user trains.

When the workout is completed, the backend groups those results by exercise and compares the actual performance against the targets defined by the workout.

Successful sessions move the progression counter upward. Unsuccessful sessions move it downward.

When the threshold is reached in either direction, the target weight for the exercise is adjusted and the counter resets.

That means the next time the user starts the workout, the new target is already part of the routine.



From MVP to product

NEW FEATURES AND PAGES

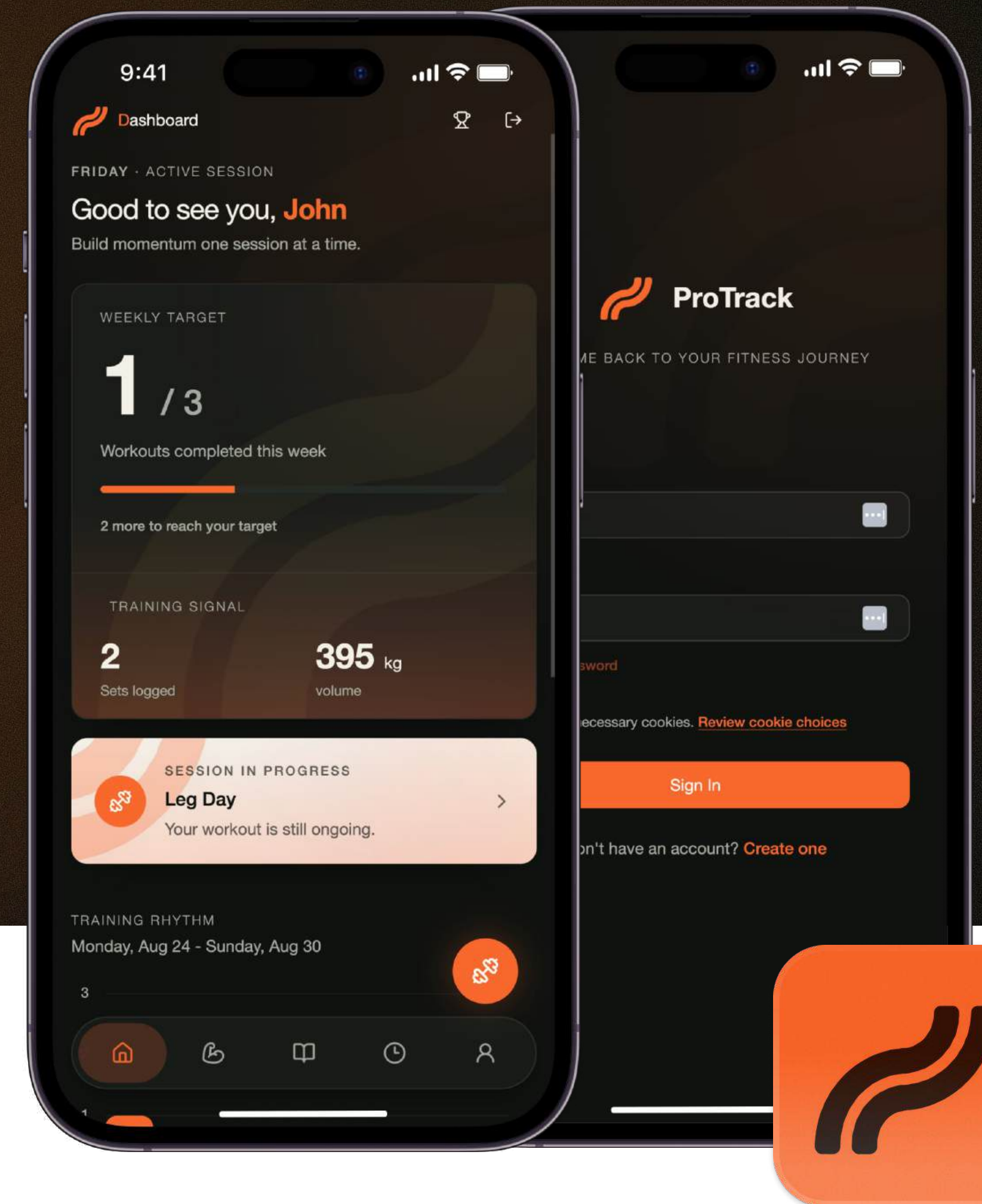
The first milestone was getting the core loop working reliably: create a workout, complete it, store every set, and update progression.

At that point the application technically solved the original problem.

However, that felt more like an MVP than an actual app I would use.

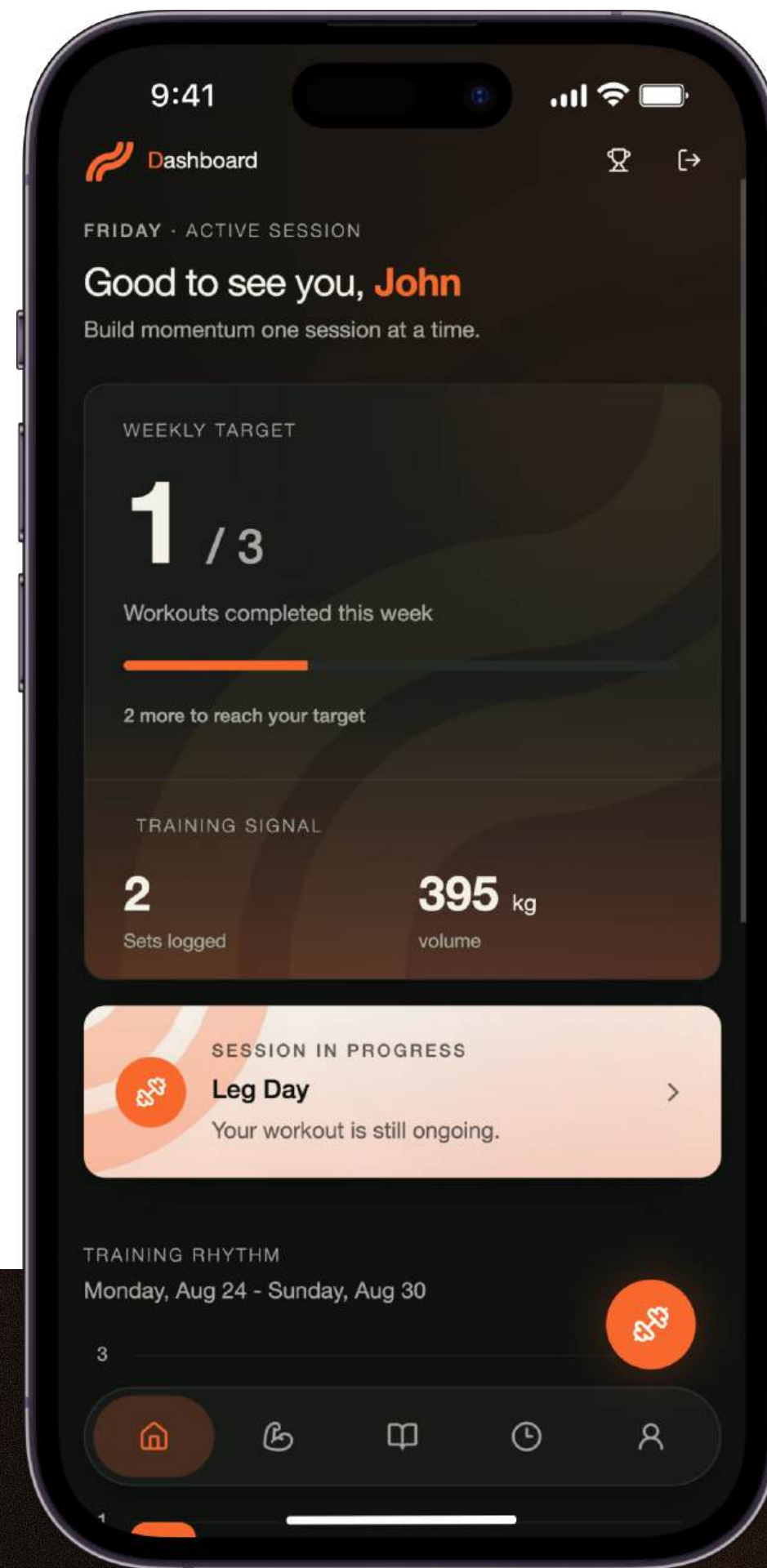
So the next phase was about turning that working system into something that felt like a complete product.

Since I had so much fun seeing all that finally working seamlessly, I started adding a few supporting areas users would expect from a real fitness application, while making sure they still connected back to the same workout data.

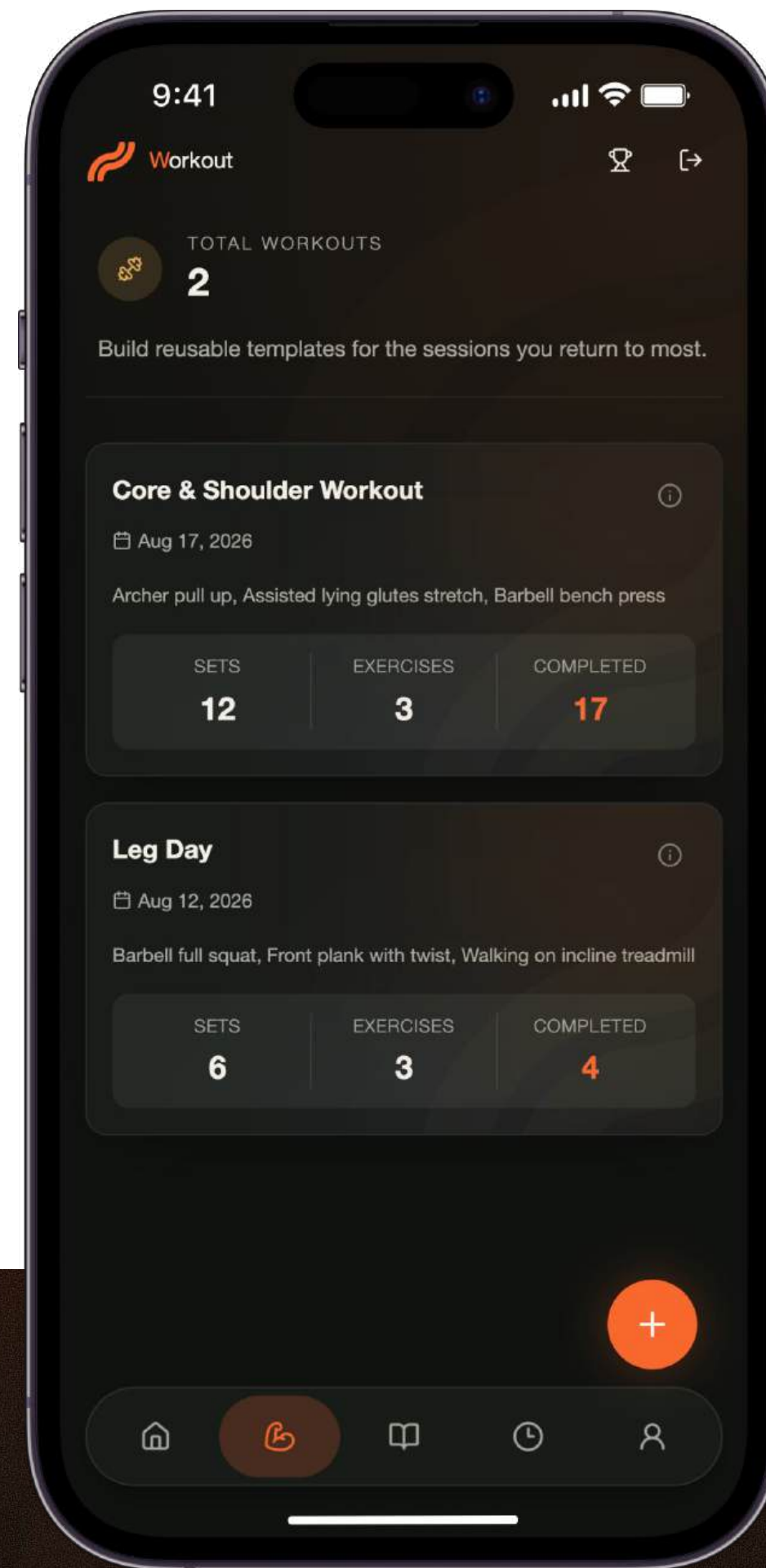


From MVP to product

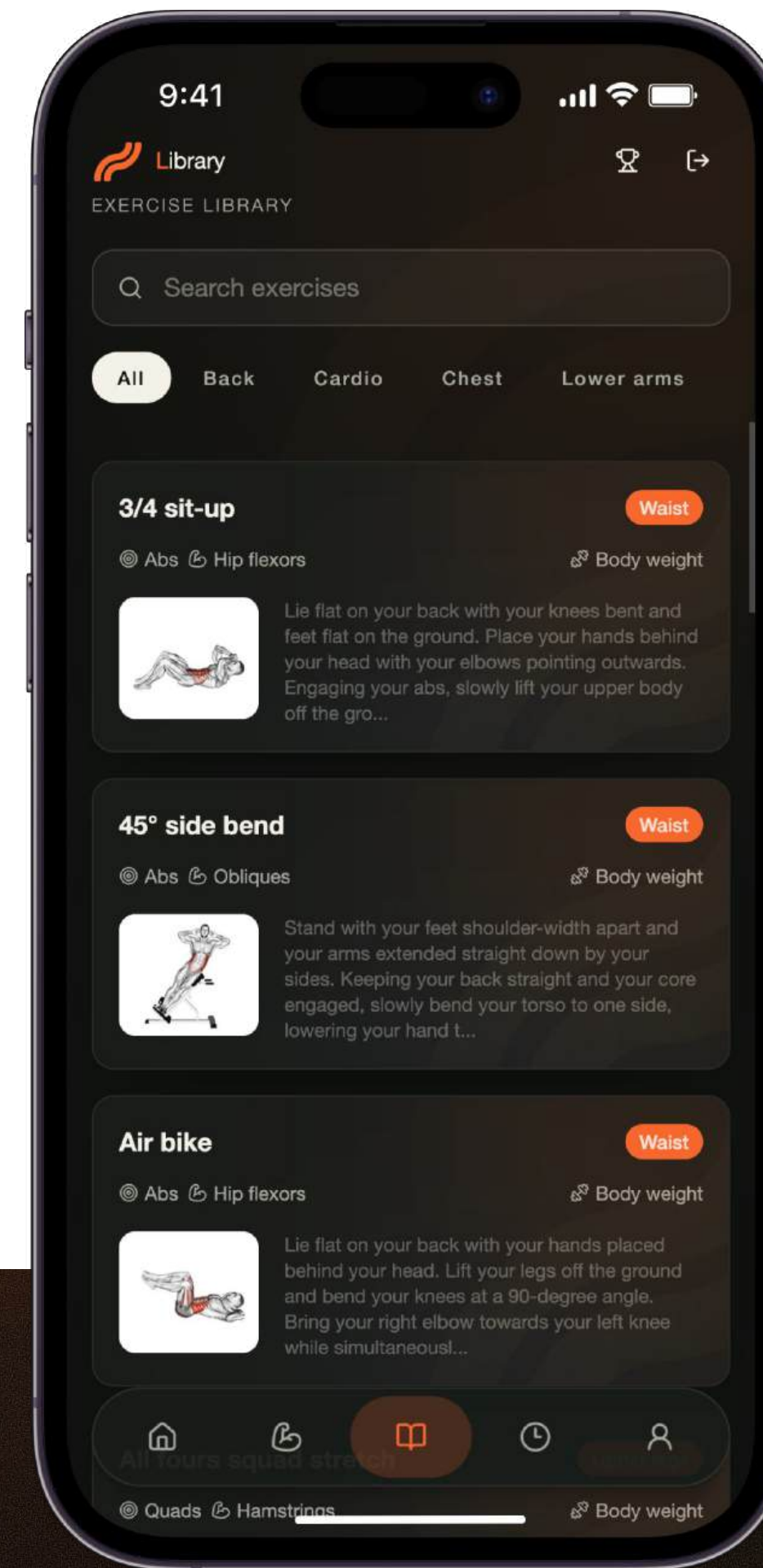
APP SECTIONS SHOWCASE



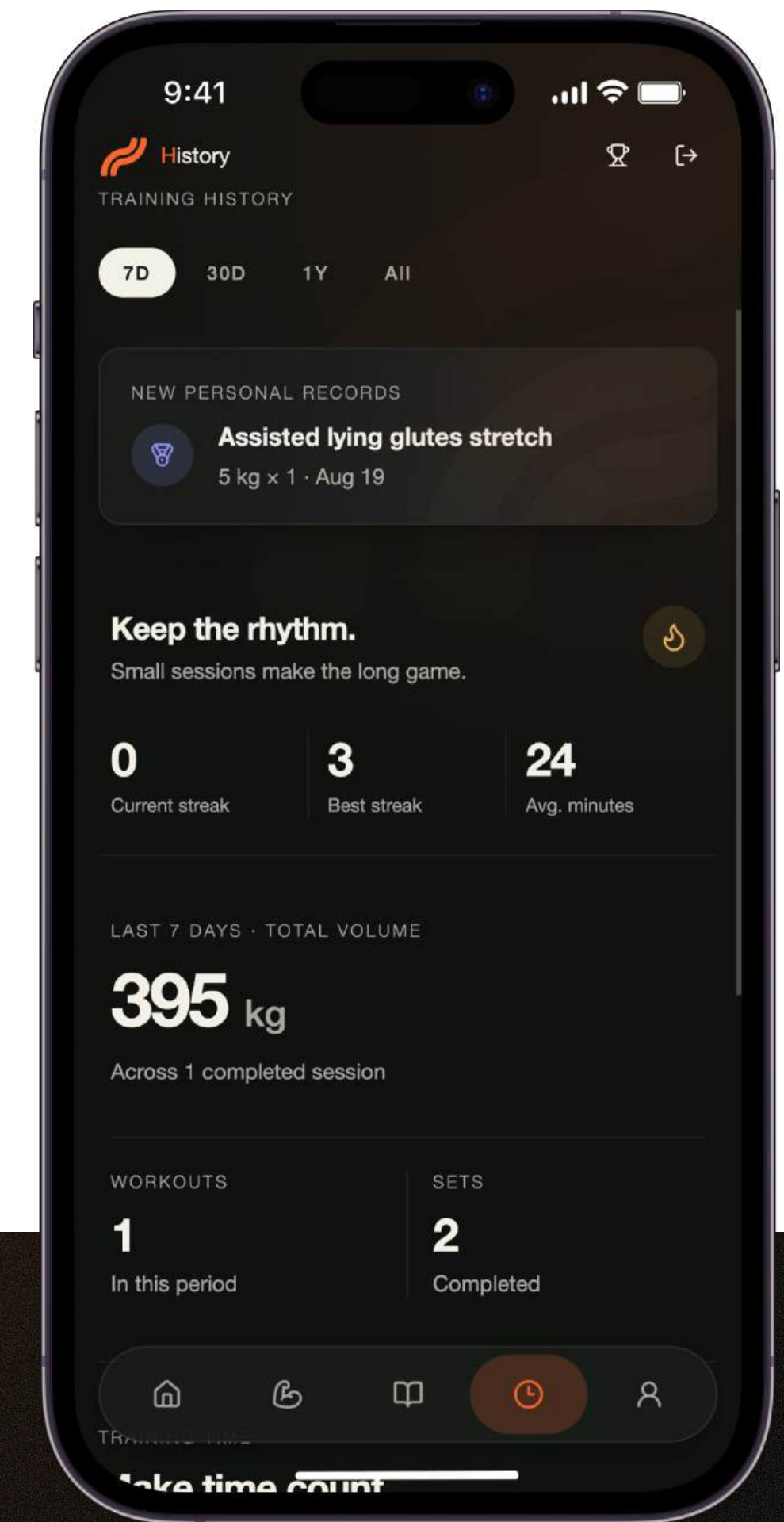
DASHBOARD



WORKOUTS



LIBRARY



HISTORY

History & Statistics

HOW STORED WORKOUT DATA BECOMES USEFUL

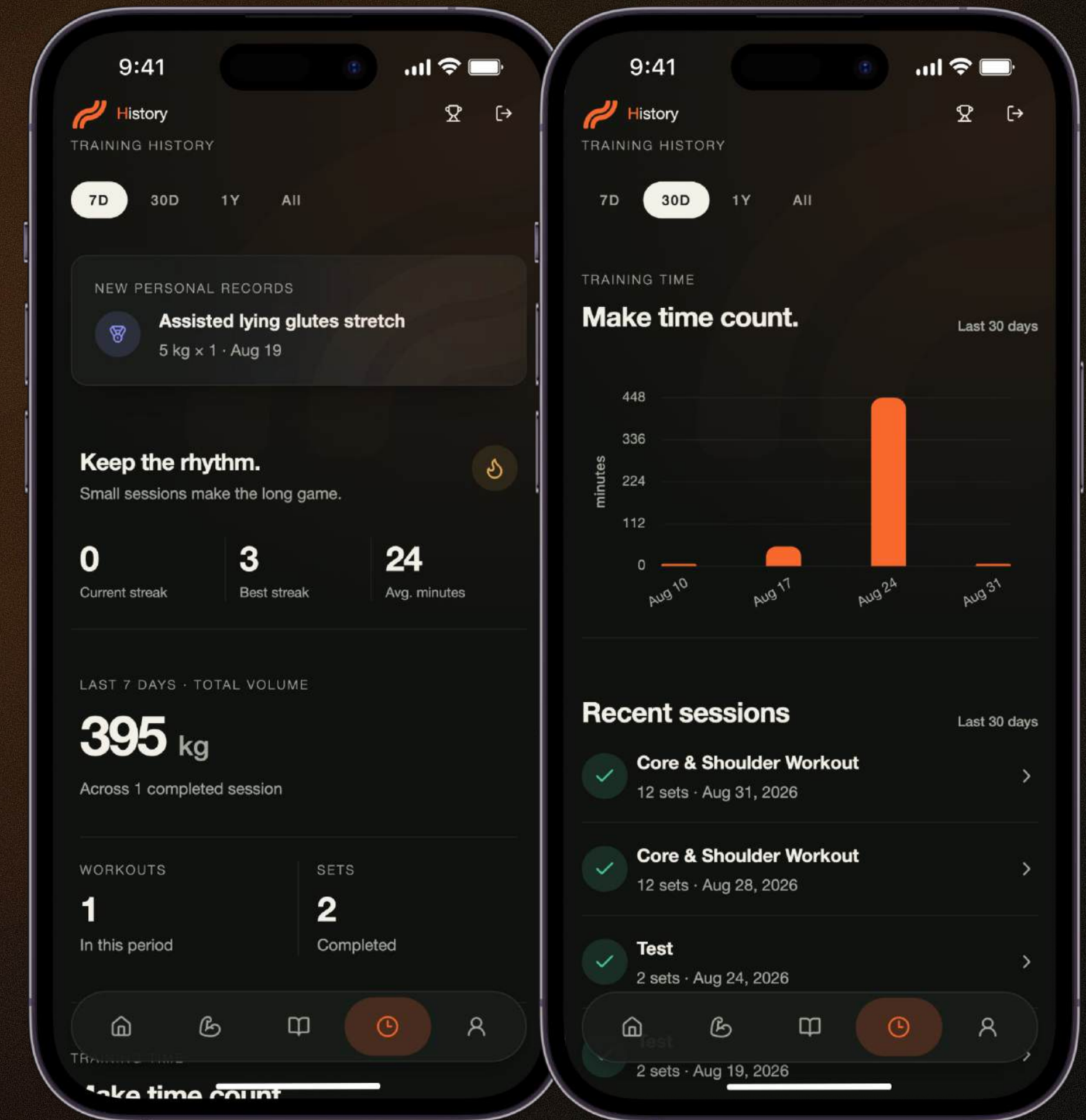
Because every workout and every set is stored, ProTrack can use the same data for longer-term analysis.

The History section lets the user look back at previous sessions, while Statistics gives a broader view of their training behavior.

Instead of showing everything on the Dashboard, I separated immediate information from deeper analytics.

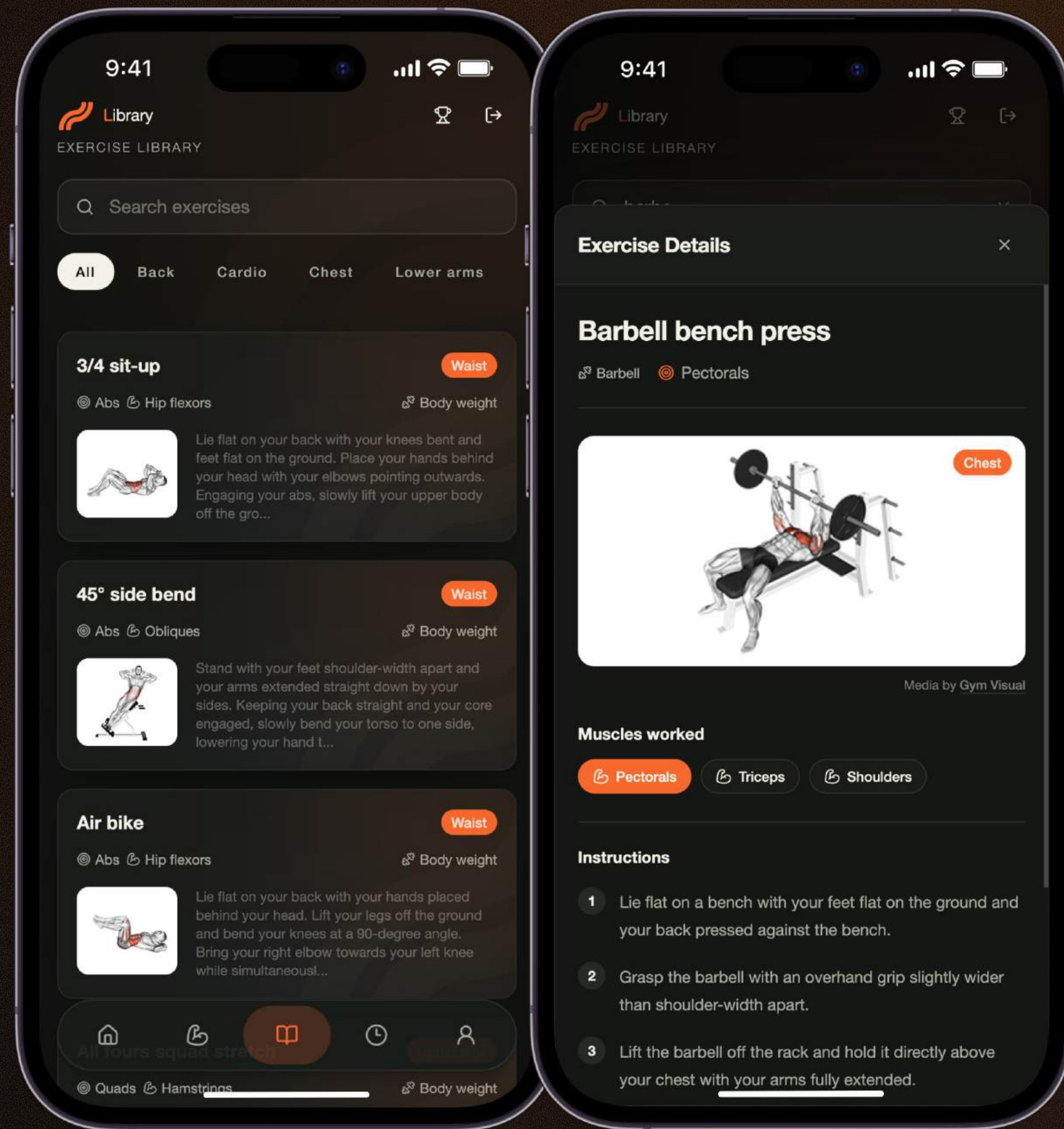
It also shows recent session more extensively and the Current PR (if any) for a related exercise.

If the Dashboard helps answer 'what should I do now?' History and Statistics answer 'what has been happening over time?'



Exercise library

A RELIABLE SOURCE OF INFORMATION



For the exercise content itself, I didn't want to spend the project manually creating an exercise database. Nor did I wanted to leave users going through the whole process of creating each exercise sheet.

In fact, I found an open source [exercise library on GitHub](#) containing the information and visual assets I needed, including animations, instructions and categorization of more than **1300 exercises**.

The interesting part for me was then integrating that dataset into the product and building a coherent UI.

I built the search, filtering, exercise details and workout selection experience around it, and also used its categorization to determine how different exercises should behave during an active workout.

The extensive **filtering** based on different attributes allowed me to build a useful filter bar to select the right muscle groups to train.

Achievements & Goals

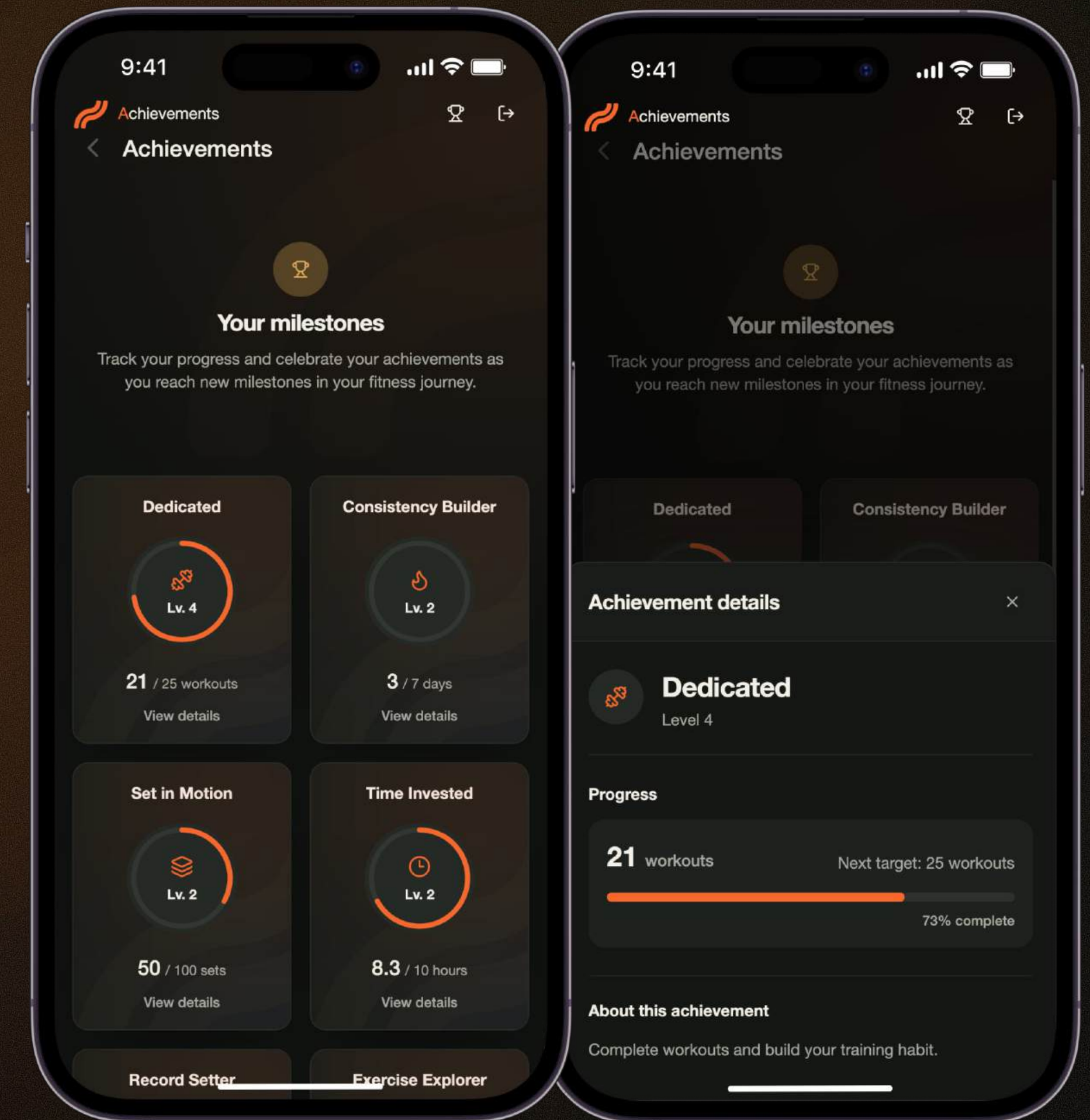
HOW TO MOTIVATE CONSISTENCY

Once the core experience was working, I also experimented with lightweight **gamification**.

The goal was to make progress that might otherwise go unnoticed more visible and **engaging**.

Achievements can reward things such as completing the first workout, maintaining consistency, reaching training volume milestones or setting personal records, they have different levels that progress as the previous threshold is reached.

I believe it can become another layer of feedback without interfering with the main training flow.



What now?

FEW IDEAS FOR THE NEXT VERSION OF **PROTRACK**

AI TRAINING COMPANION

A coach with access to the user's actual training history that could access info about user's last training session as well as other history information and provide suggestions based on it.

GLOBAL PRODUCT SUPPORT

Multilingual exercise names and instructions support. Configurable in the profile section.

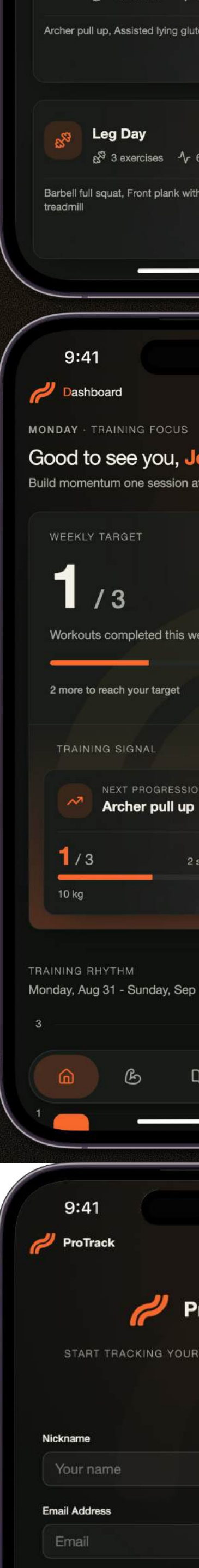
BACKGROUND NOTIFICATIONS

Background push notification system to remind users of their weekly goals, active sessions and achievements, making the app even more interactive.

SMART WORKOUT GENERATION

Generating workouts based on constraints such as available equipment, available time and recent training history.

As an ongoing project, my intent is to continuously ship updates and features while fixing bugs identified through feedback and personal use. Since I chose to keep ProTrack as a **PWA**, the deployment process should remain fast and straightforward.

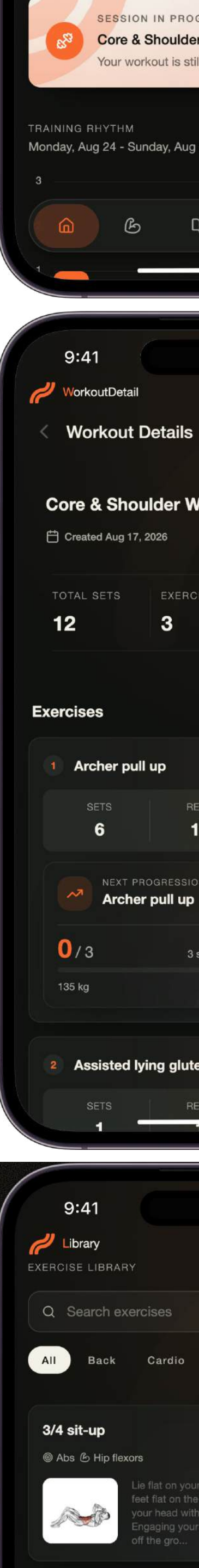


Conclusions & Takeaways

WHAT BUILDING **PROTRACK** TAUGHT ME

I had so much fun working on ProTrack, and the passion I put into it and the things I learned are basically endless.

- One of the first and most important things I struggled with was understanding **how much a data model can influence the final system design of a product**. Initially, a workout seems like one thing, but the application needs to distinguish between the workout someone plans, the session they perform, and the individual sets they complete. Once I separated those concepts, features like history, resuming sessions and progressive overload became much cleaner to implement.
- I also learned that **reliability** is part of UX. If someone is halfway through a workout and refreshes the application, losing all their progress is a user-experience failure just as much as a technical one. That's why sets are persisted while the workout happens and sessions can be resumed.

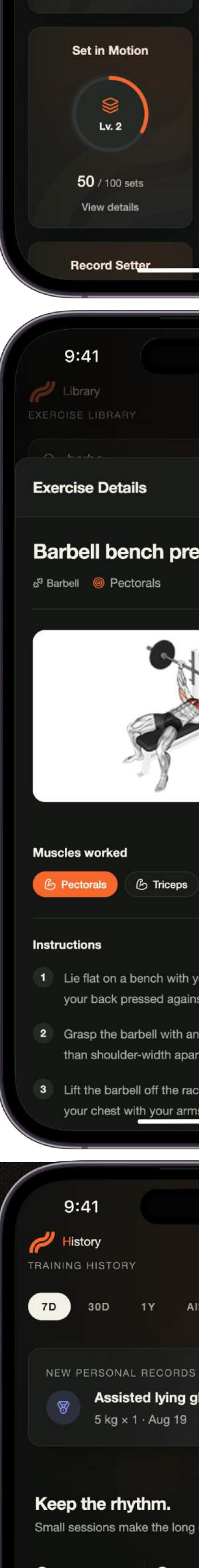


Conclusions & Takeaways

WHAT BUILDING **PROTRACK** TAUGHT ME

- The progressive overload mechanic taught me another important lesson about shipping features and displaying results: the logic behind a feature can be complex, but **that complexity shouldn't reach the end user**. The backend might be tracking thresholds and counters, while the interface only needs to communicate the progress made and the next step.
- Overall, these features showed me that frontend and backend aren't really independent when you're building a product. Even one small progression card depends on UI design, frontend state, API contracts, backend logic and persistent data working together.

The main takeaway from this project is a shift in how I think about development: not just as moving data from a database to an interface, but as the **complete path** from a user action, through the system, and back to feedback that is actually useful. This feels even more important today, when generative AI makes the “how” less central and puts more value on being able to understand and picture the **full scope** of a product.





TRY THE
PROTRACK APP

DESIGNED & DEVELOPED BY

Manuel La Porta

 [linkedin.com/in/manulaporta](https://www.linkedin.com/in/manulaporta)

 github.com/Manulapo